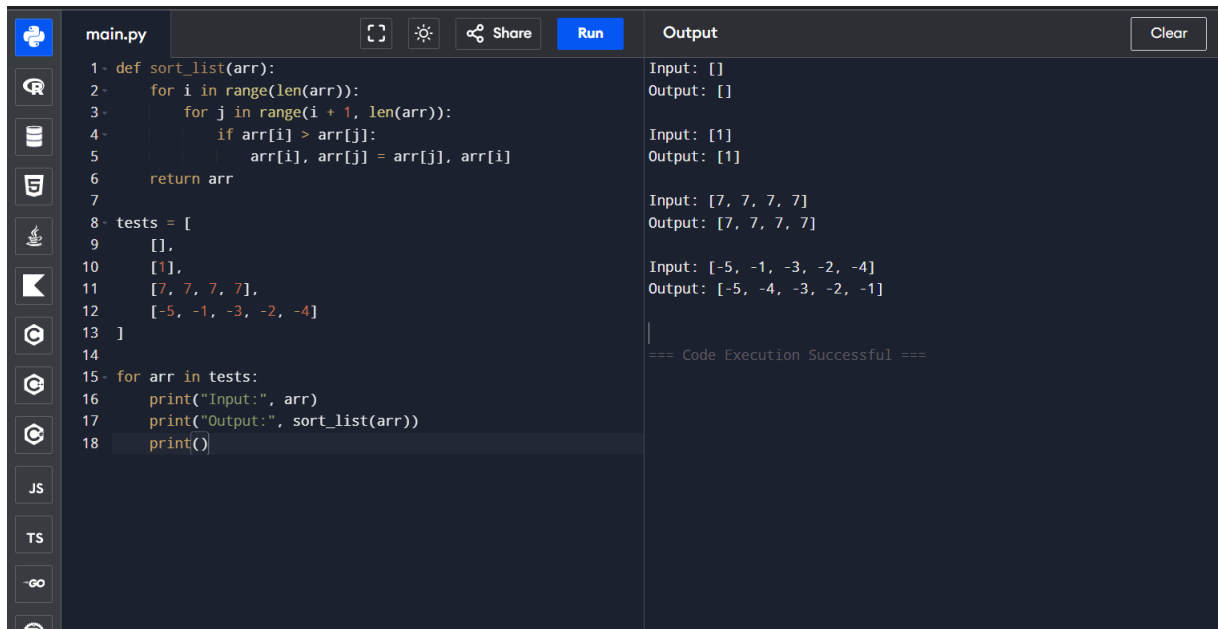


TOPIC 2 : BRUTE FORCE

1. List Handling and Sorting



The screenshot shows a code editor with a file named `main.py`. The code implements a bubble sort function `sort_list(arr)` and tests it with several input lists. The output shows the sorted lists for each input.

```
1 def sort_list(arr):
2     for i in range(len(arr)):
3         for j in range(i + 1, len(arr)):
4             if arr[i] > arr[j]:
5                 arr[i], arr[j] = arr[j], arr[i]
6     return arr
7
8 tests = [
9     [],
10    [1],
11    [7, 7, 7, 7],
12    [-5, -1, -3, -2, -4]
13 ]
14
15 for arr in tests:
16     print("Input:", arr)
17     print("Output:", sort_list(arr))
18     print()
```

Output:

```
Input: []
Output: []

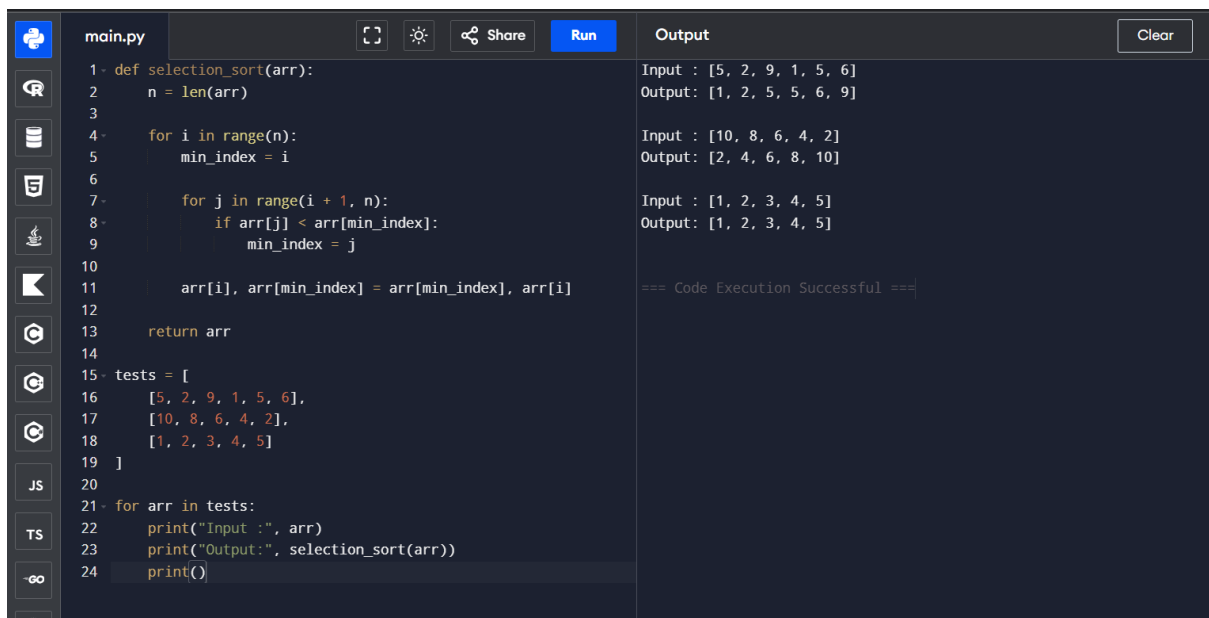
Input: [1]
Output: [1]

Input: [7, 7, 7, 7]
Output: [7, 7, 7, 7]

Input: [-5, -1, -3, -2, -4]
Output: [-5, -4, -3, -2, -1]

=== Code Execution Successful ===
```

2. Selection Sort



The screenshot shows a code editor with a file named `main.py`. The code implements a selection sort function `selection_sort(arr)` and tests it with several input lists. The output shows the sorted lists for each input.

```
1 def selection_sort(arr):
2     n = len(arr)
3
4     for i in range(n):
5         min_index = i
6
7         for j in range(i + 1, n):
8             if arr[j] < arr[min_index]:
9                 min_index = j
10
11        arr[i], arr[min_index] = arr[min_index], arr[i]
12
13    return arr
14
15 tests = [
16     [5, 2, 9, 1, 5, 6],
17     [10, 8, 6, 4, 2],
18     [1, 2, 3, 4, 5]
19 ]
20
21 for arr in tests:
22     print("Input :", arr)
23     print("Output:", selection_sort(arr))
24     print()
```

Output:

```
Input : [5, 2, 9, 1, 5, 6]
Output: [1, 2, 5, 5, 6, 9]

Input : [10, 8, 6, 4, 2]
Output: [2, 4, 6, 8, 10]

Input : [1, 2, 3, 4, 5]
Output: [1, 2, 3, 4, 5]

=== Code Execution Successful ===
```

3. Optimized Bubble Sort



```
main.py  [Full Screen] [Settings] [Share] [Run] [Clear]

1- def bubble_sort(arr):
2-     n = len(arr)
3-
4-     for i in range(n - 1):
5-         swapped = False
6-
7-         for j in range(n - 1 - i):
8-             if arr[j] > arr[j + 1]:
9-                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
10-                swapped = True
11-
12-         if not swapped:
13-             break
14-
15-     return arr
16-
17- tests = [
18-     [5, 2, 9, 1, 5, 6],
19-     [1, 2, 3, 4, 5],
20-     [5, 4, 3, 2, 1]
21- ]
22-
23- for arr in tests:
24-     print("Input :", arr)
25-     print("Output:", bubble_sort(arr))
26-     print()
```

Output

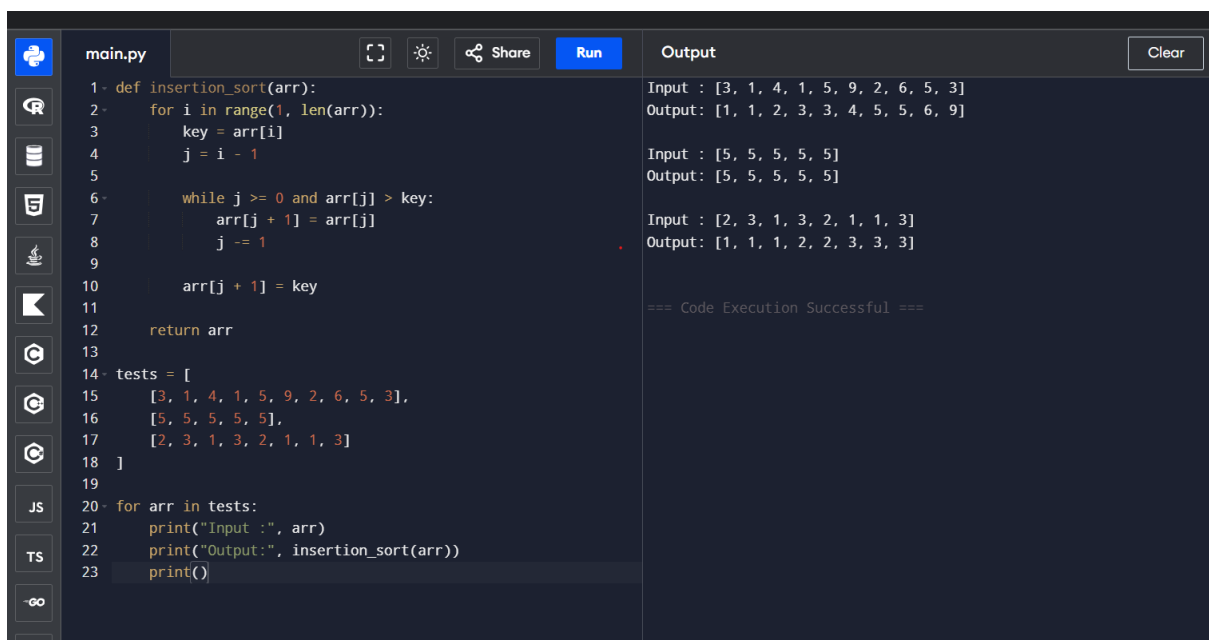
Input : [5, 2, 9, 1, 5, 6]
Output: [1, 2, 5, 5, 6, 9]

Input : [1, 2, 3, 4, 5]
Output: [1, 2, 3, 4, 5]

Input : [5, 4, 3, 2, 1]
Output: [1, 2, 3, 4, 5]

=== Code Execution Successful ===

4. Insertion Sort with Duplicates



```
main.py  [Full Screen] [Settings] [Share] [Run] [Clear]

1- def insertion_sort(arr):
2-     for i in range(1, len(arr)):
3-         key = arr[i]
4-         j = i - 1
5-
6-         while j >= 0 and arr[j] > key:
7-             arr[j + 1] = arr[j]
8-             j -= 1
9-
10-        arr[j + 1] = key
11-
12-    return arr
13-
14- tests = [
15-     [3, 1, 4, 1, 5, 9, 2, 6, 5, 3],
16-     [5, 5, 5, 5, 5],
17-     [2, 3, 1, 3, 2, 1, 1, 3]
18- ]
19-
20- for arr in tests:
21-     print("Input :", arr)
22-     print("Output:", insertion_sort(arr))
23-     print()
```

Output

Input : [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]
Output: [1, 1, 2, 3, 3, 4, 5, 5, 6, 9]

Input : [5, 5, 5, 5, 5]
Output: [5, 5, 5, 5, 5]

Input : [2, 3, 1, 3, 2, 1, 1, 3]
Output: [1, 1, 1, 2, 2, 3, 3, 3]

=== Code Execution Successful ===

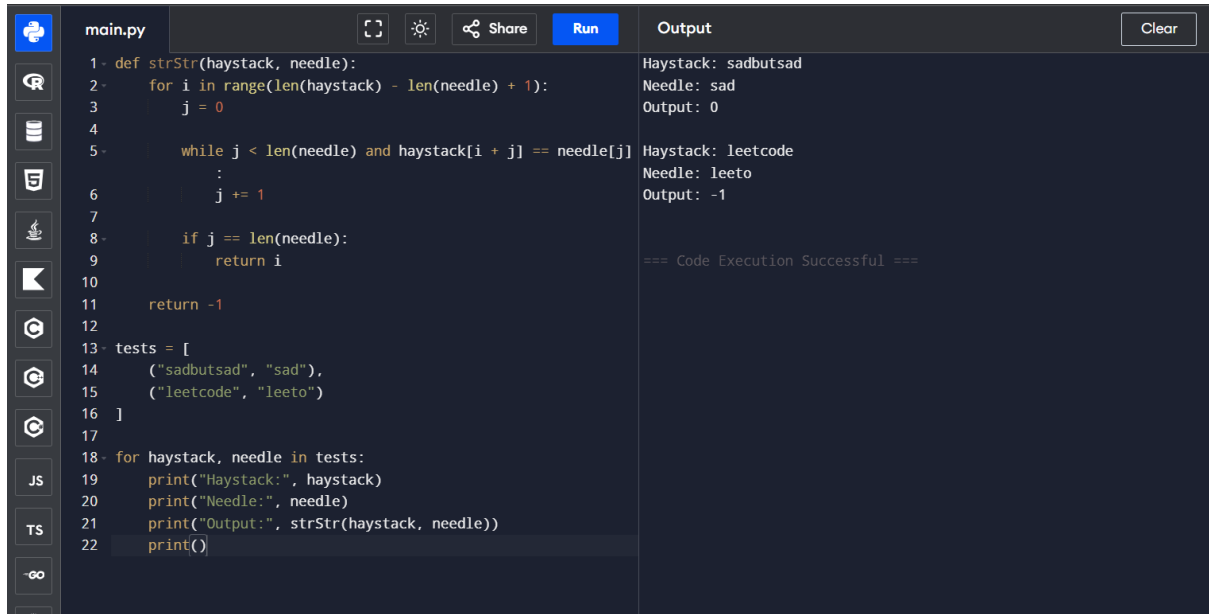
5. Kth Missing Positive Number

main.py	Run	Output
<pre>1 def findKthMissing(arr, k): 2 missing = 0 3 num = 1 4 i = 0 5 6 while missing < k: 7 if i < len(arr) and arr[i] == num: 8 i += 1 9 else: 10 missing += 1 11 if missing == k: 12 return num 13 num += 1 14 15 tests = [16 ([2, 3, 4, 7, 11], 5), 17 ([1, 2, 3, 4], 2) 18] 19 20 for arr, k in tests: 21 print("Input:", arr, "k =", k) 22 print("Output:", findKthMissing(arr, k)) 23 print()</pre>		Input: [2, 3, 4, 7, 11] k = 5 Output: 9 Input: [1, 2, 3, 4] k = 2 Output: 6 === Code Execution Successful ===

6. Peak Element Search

main.py	Run	Output
<pre>1 def findPeak(nums): 2 left = 0 3 right = len(nums) - 1 4 5 while left < right: 6 mid = (left + right) // 2 7 8 if nums[mid] < nums[mid + 1]: 9 left = mid + 1 10 else: 11 right = mid 12 13 return left 14 15 tests = [16 [1, 2, 3, 1], 17 [1, 2, 1, 3, 5, 6, 4] 18] 19 20 for nums in tests: 21 index = findPeak(nums) 22 print("Input:", nums) 23 print("Peak Index:", index) 24 print("Peak Element:", nums[index]) 25 print()</pre>		Input: [1, 2, 3, 1] Peak Index: 2 Peak Element: 3 Input: [1, 2, 1, 3, 5, 6, 4] Peak Index: 5 Peak Element: 6 === Code Execution Successful ===

7.String Search



The screenshot shows a code editor with a file named `main.py`. The code defines a function `strStr` that searches for a needle in a haystack. It uses a nested loop to compare characters. The code also includes test cases and prints the results.

```
1 def strStr(haystack, needle):
2     for i in range(len(haystack) - len(needle) + 1):
3         j = 0
4
5         while j < len(needle) and haystack[i + j] == needle[j]:
6             j += 1
7
8         if j == len(needle):
9             return i
10
11     return -1
12
13 tests = [
14     ("sadbutsad", "sad"),
15     ("leetcode", "leeto")
16 ]
17
18 for haystack, needle in tests:
19     print("Haystack:", haystack)
20     print("Needle:", needle)
21     print("Output:", strStr(haystack, needle))
22     print()
```

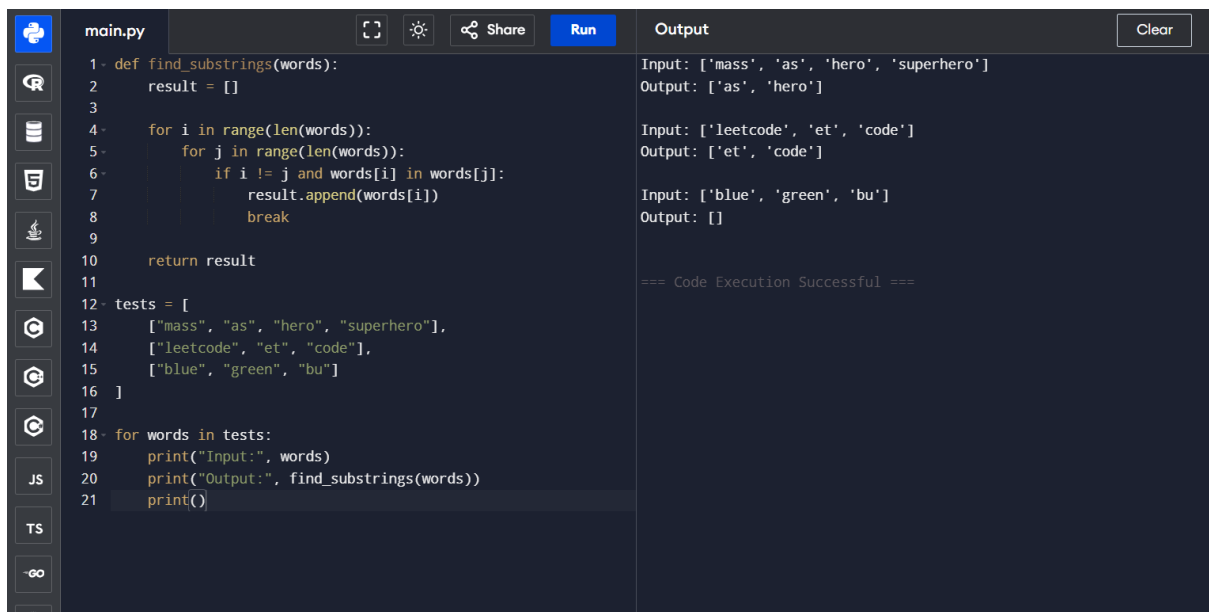
The output shows the results of the string search for two test cases:

```
Haystack: sadbutsad
Needle: sad
Output: 0

Haystack: leetcode
Needle: leeto
Output: -1

=== Code Execution Successful ===
```

8. Substring Detection



The screenshot shows a code editor with a file named `main.py`. The code defines a function `find_substrings` that finds all substrings of a given list of words. It uses a nested loop to generate all possible substrings. The code also includes test cases and prints the results.

```
1 def find_substrings(words):
2     result = []
3
4     for i in range(len(words)):
5         for j in range(len(words)):
6             if i != j and words[i] in words[j]:
7                 result.append(words[i])
8                 break
9
10    return result
11
12 tests = [
13     ["mass", "as", "hero", "superhero"],
14     ["leetcode", "et", "code"],
15     ["blue", "green", "bu"]
16 ]
17
18 for words in tests:
19     print("Input:", words)
20     print("Output:", find_substrings(words))
21     print()
```

The output shows the results of the substring detection for three test cases:

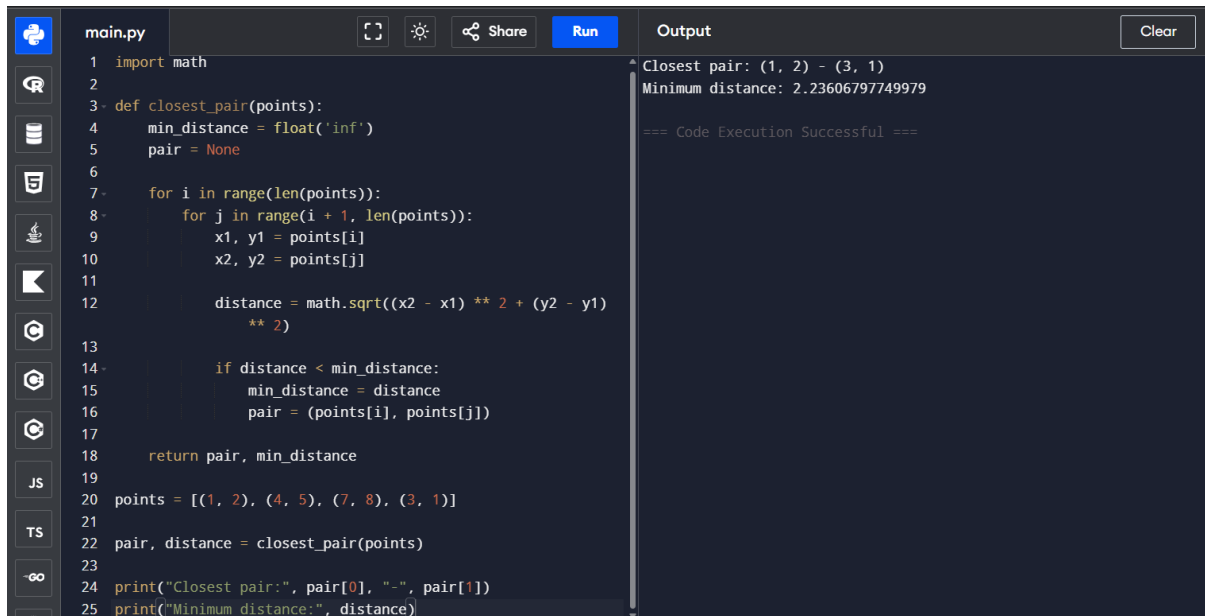
```
Input: ['mass', 'as', 'hero', 'superhero']
Output: ['as', 'hero']

Input: ['leetcode', 'et', 'code']
Output: ['et', 'code']

Input: ['blue', 'green', 'bu']
Output: []

=== Code Execution Successful ===
```

9. Closest Pair of Points

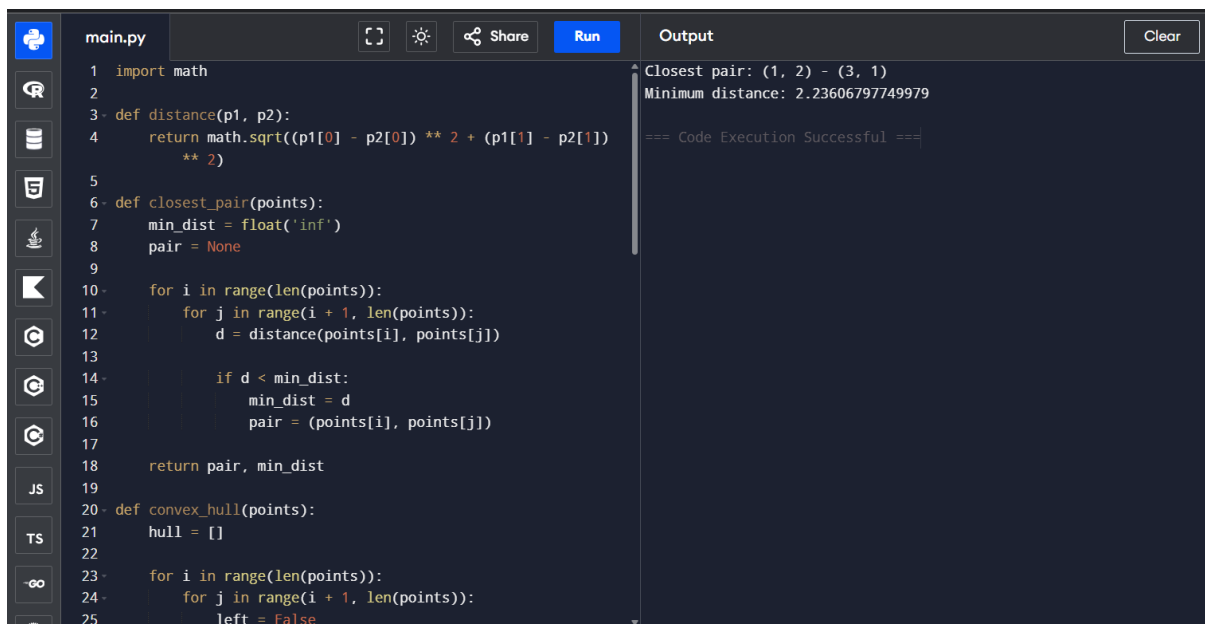


```
1 import math
2
3 def closest_pair(points):
4     min_distance = float('inf')
5     pair = None
6
7     for i in range(len(points)):
8         for j in range(i + 1, len(points)):
9             x1, y1 = points[i]
10            x2, y2 = points[j]
11
12            distance = math.sqrt((x2 - x1) ** 2 + (y2 - y1)
13                                ** 2)
14
15            if distance < min_distance:
16                min_distance = distance
17                pair = (points[i], points[j])
18
19    return pair, min_distance
20
21 points = [(1, 2), (4, 5), (7, 8), (3, 1)]
22
23 pair, distance = closest_pair(points)
24
25 print("Closest pair:", pair[0], "-", pair[1])
26 print("Minimum distance:", distance)
```

Output

```
Closest pair: (1, 2) - (3, 1)
Minimum distance: 2.23606797749979
=== Code Execution Successful ===
```

10. Brute Force Convex Hull

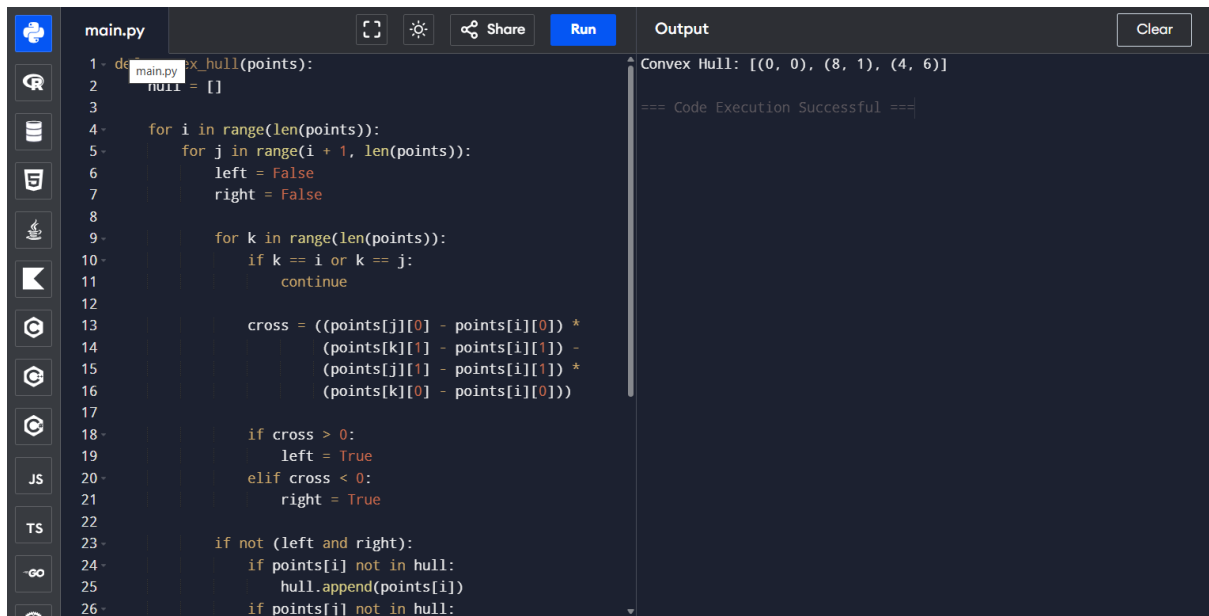


```
1 import math
2
3 def distance(p1, p2):
4     return math.sqrt((p1[0] - p2[0]) ** 2 + (p1[1] - p2[1])
5                     ** 2)
6
7 def closest_pair(points):
8     min_dist = float('inf')
9     pair = None
10
11    for i in range(len(points)):
12        for j in range(i + 1, len(points)):
13            d = distance(points[i], points[j])
14
15            if d < min_dist:
16                min_dist = d
17                pair = (points[i], points[j])
18
19    return pair, min_dist
20
21 def convex_hull(points):
22     hull = []
23
24     for i in range(len(points)):
25         for j in range(i + 1, len(points)):
26             left = False
```

Output

```
Closest pair: (1, 2) - (3, 1)
Minimum distance: 2.23606797749979
=== Code Execution Successful ===
```

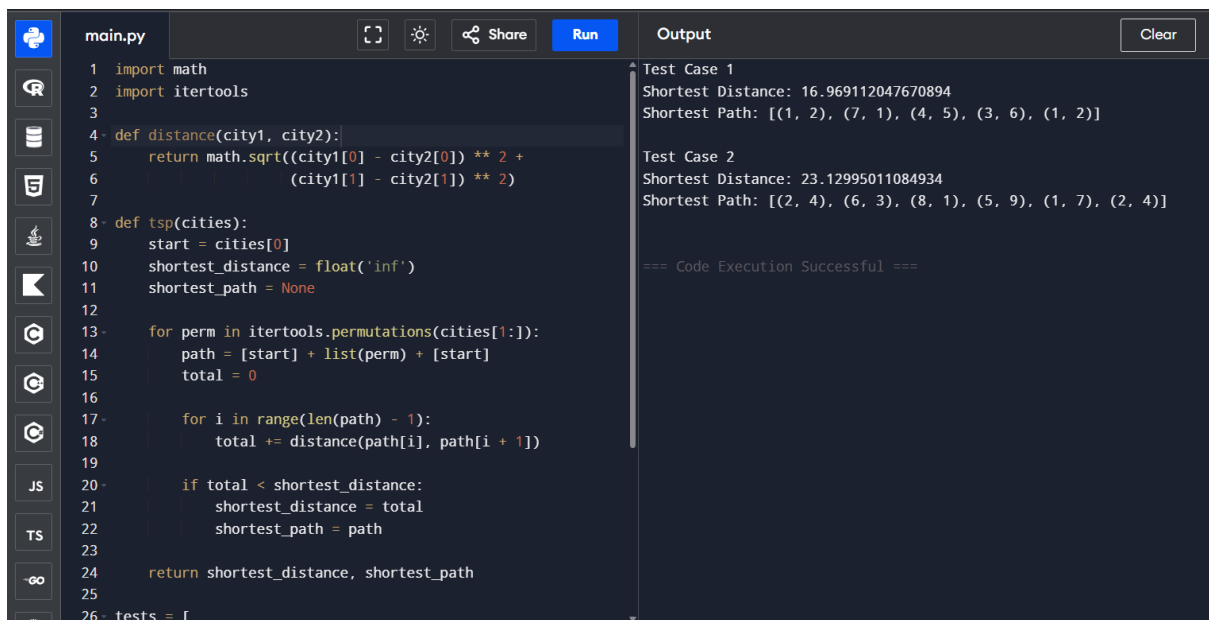
11.Convex Hull



The screenshot shows a code editor with a file named `main.py`. The code implements a Convex Hull algorithm. It starts by defining a function `convex_hull(points)` which initializes an empty list `hull`. It then iterates over all points, checking if they are on the left or right side of the line segment formed by the previous two points in the hull. The cross product is used to determine the orientation. Points that are on the left are added to the hull. The output shows the resulting convex hull points: `Convex Hull: [(0, 0), (8, 1), (4, 6)]` and a success message: `=== Code Execution Successful ===`.

```
1 def convex_hull(points):
2     hull = []
3
4     for i in range(len(points)):
5         for j in range(i + 1, len(points)):
6             left = False
7             right = False
8
9             for k in range(len(points)):
10                if k == i or k == j:
11                    continue
12
13                cross = ((points[j][0] - points[i][0]) *
14                        (points[k][1] - points[i][1]) -
15                        (points[j][1] - points[i][1]) *
16                        (points[k][0] - points[i][0]))
17
18                if cross > 0:
19                    left = True
20                elif cross < 0:
21                    right = True
22
23                if not (left and right):
24                    if points[i] not in hull:
25                        hull.append(points[i])
26                    if points[j] not in hull:
```

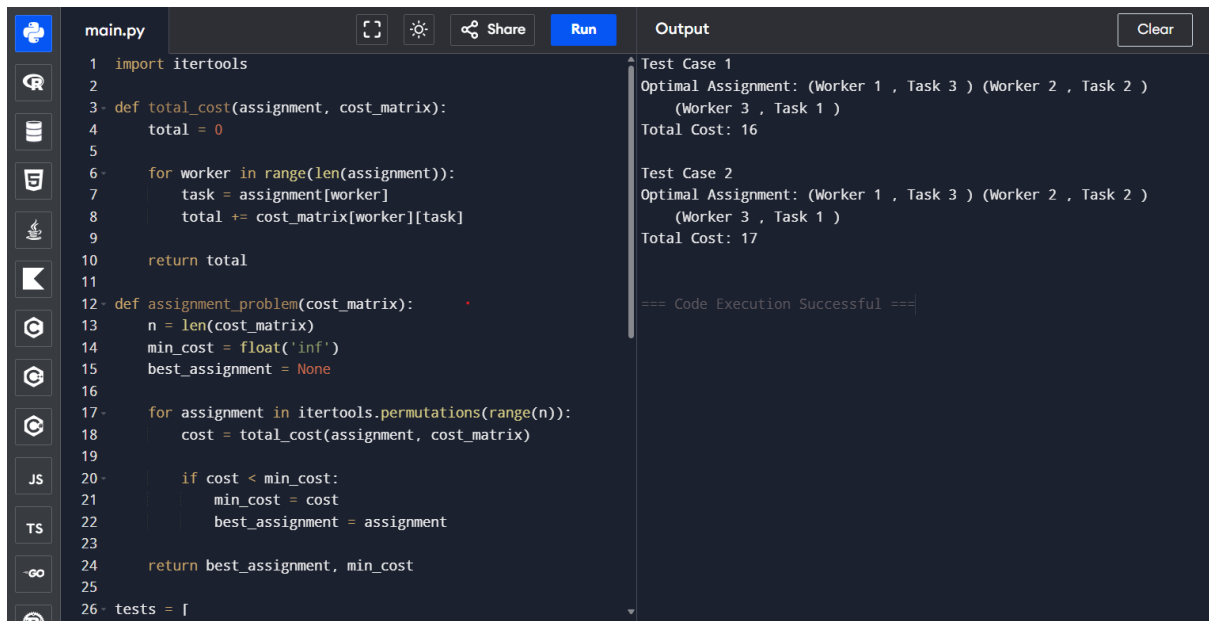
12.Travelling Salesman Problem (TSP)



The screenshot shows a code editor with a file named `main.py`. The code implements a solution for the Travelling Salesman Problem (TSP) using a brute-force approach. It defines a `distance` function to calculate the distance between two cities. The `tsp` function iterates over all possible permutations of cities, calculates the total distance for each path, and keeps track of the shortest distance and path. The output shows the results for two test cases: `Test Case 1` with a shortest distance of `16.969112047670894` and a shortest path of `[(1, 2), (7, 1), (4, 5), (3, 6), (1, 2)]`, and `Test Case 2` with a shortest distance of `23.12995011084934` and a shortest path of `[(2, 4), (6, 3), (8, 1), (5, 9), (1, 7), (2, 4)]`. The output also includes a success message: `=== Code Execution Successful ===`.

```
1 import math
2 import itertools
3
4 def distance(city1, city2):
5     return math.sqrt((city1[0] - city2[0]) ** 2 +
6                     (city1[1] - city2[1]) ** 2)
7
8 def tsp(cities):
9     start = cities[0]
10    shortest_distance = float('inf')
11    shortest_path = None
12
13    for perm in itertools.permutations(cities[1:]):
14        path = [start] + list(perm) + [start]
15        total = 0
16
17        for i in range(len(path) - 1):
18            total += distance(path[i], path[i + 1])
19
20        if total < shortest_distance:
21            shortest_distance = total
22            shortest_path = path
23
24    return shortest_distance, shortest_path
25
26 tests = [
```

13. Assignment Problem



```
main.py  [Full Screen] [Theme] [Share] [Run] [Clear]

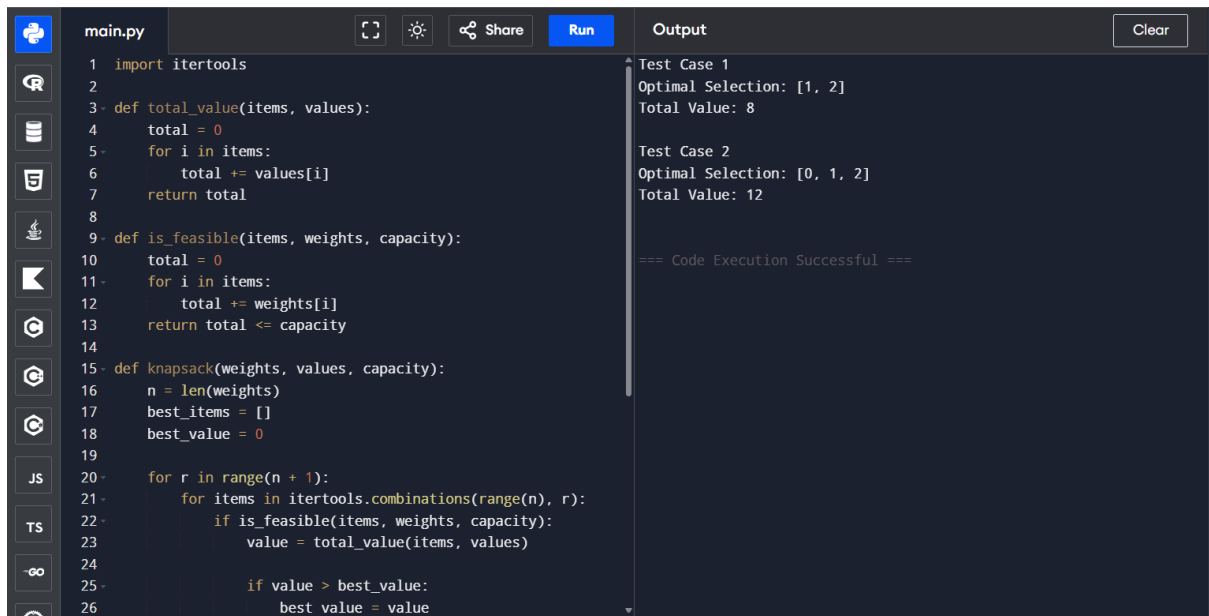
1 import itertools
2
3 def total_cost(assignment, cost_matrix):
4     total = 0
5
6     for worker in range(len(assignment)):
7         task = assignment[worker]
8         total += cost_matrix[worker][task]
9
10    return total
11
12 def assignment_problem(cost_matrix):
13     n = len(cost_matrix)
14     min_cost = float('inf')
15     best_assignment = None
16
17     for assignment in itertools.permutations(range(n)):
18         cost = total_cost(assignment, cost_matrix)
19
20         if cost < min_cost:
21             min_cost = cost
22             best_assignment = assignment
23
24     return best_assignment, min_cost
25
26 tests = [
```

Test Case 1
Optimal Assignment: (Worker 1 , Task 3) (Worker 2 , Task 2)
(Worker 3 , Task 1)
Total Cost: 16

Test Case 2
Optimal Assignment: (Worker 1 , Task 3) (Worker 2 , Task 2)
(Worker 3 , Task 1)
Total Cost: 17

=== Code Execution Successful ===

14. 0/1 Knapsack Problem



```
main.py  [Full Screen] [Theme] [Share] [Run] [Clear]

1 import itertools
2
3 def total_value(items, values):
4     total = 0
5
6     for i in items:
7         total += values[i]
8     return total
9
10 def is_feasible(items, weights, capacity):
11     total = 0
12     for i in items:
13         total += weights[i]
14     return total <= capacity
15
16 def knapsack(weights, values, capacity):
17     n = len(weights)
18     best_items = []
19     best_value = 0
20
21     for r in range(n + 1):
22         for items in itertools.combinations(range(n), r):
23             if is_feasible(items, weights, capacity):
24                 value = total_value(items, values)
25
26                 if value > best_value:
27                     best_value = value
```

Test Case 1
Optimal Selection: [1, 2]
Total Value: 8

Test Case 2
Optimal Selection: [0, 1, 2]
Total Value: 12

=== Code Execution Successful ===